

Microprocessor Systems

Lab9 - In System Programming & POV clock

Prof. Ren-Shuo Liu

NTHU EE



Outline

- In system programming
- POV clock

In System Programming (ISP)

Burner / Programmer

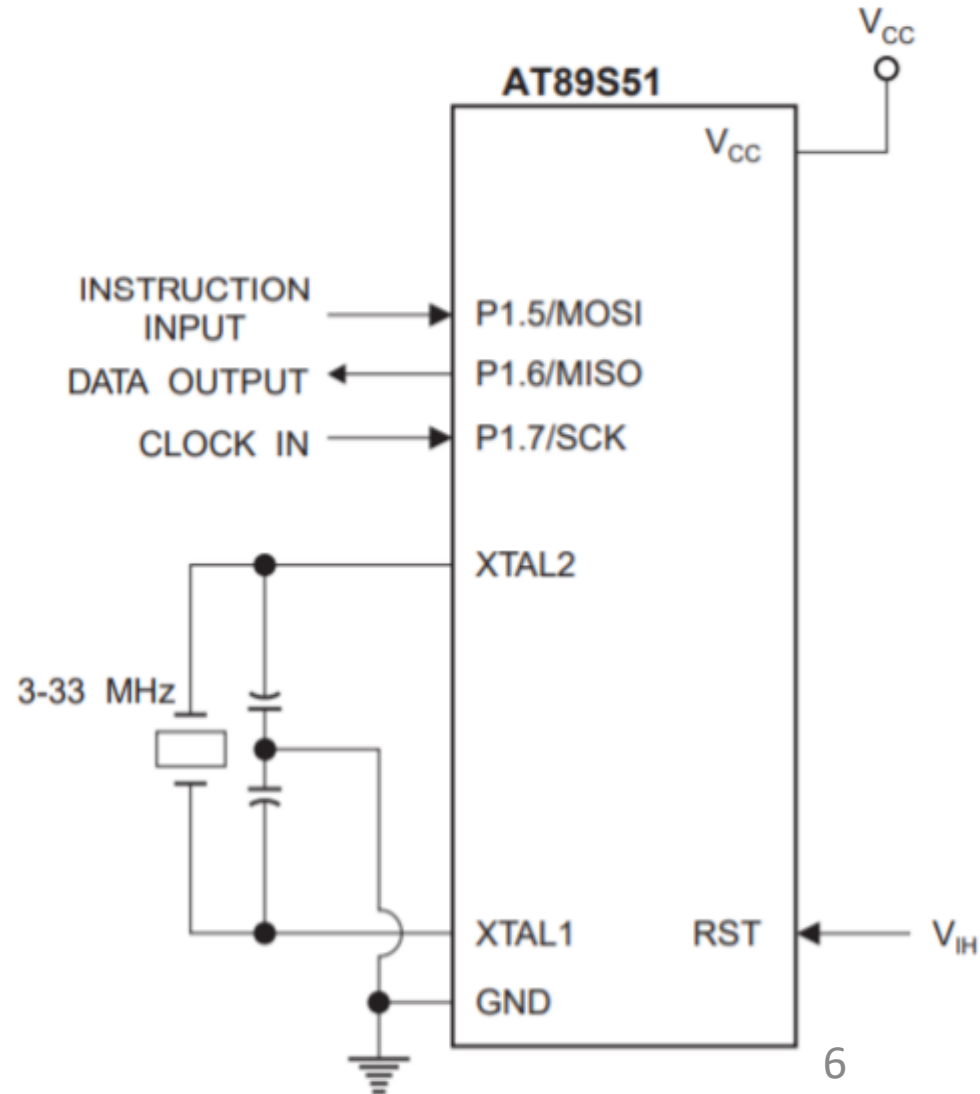
- A burner/programmer is a hardware device accompanied with software which is used to transfer the machine language code to the microcontroller/EEPROM from the PC.
- The API/software of the burner reads data from the hex file stored on the PC and feeds it into the controller's memory.
- The software transfers the data from the PC to the hardware using serial, parallel or USB port.

In System Programming (ISP)

- ISP allows microcontrollers and external memory chips can be re-programmed without being removed from the circuit board.
- For in-system programming, the following items are needed
 - A software tool running on the PC, which is able to control the programming interface via any standard port.
 - A programming adapter which allows to connect the programming interface to any standard port available at the PC, like USB.
 - A special programming interface like SPI (Serial Programming Interface).

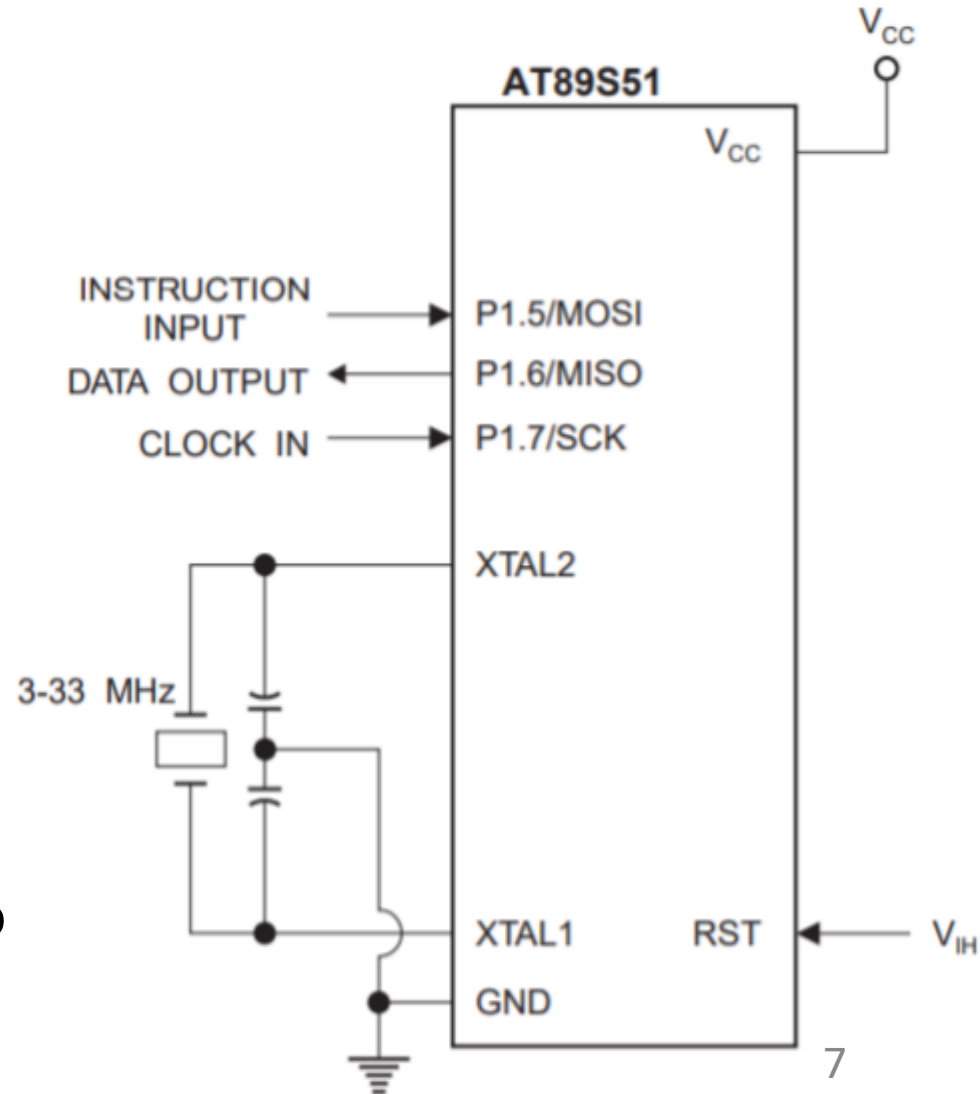
8051 Memory I/O

- Vcc: Connected to +5V supply.
- GND: Connected to ground.
- RST: It tells the target controller that it is under the programming mode.
- SCK: This pin is used to supply clock to the target microcontroller.

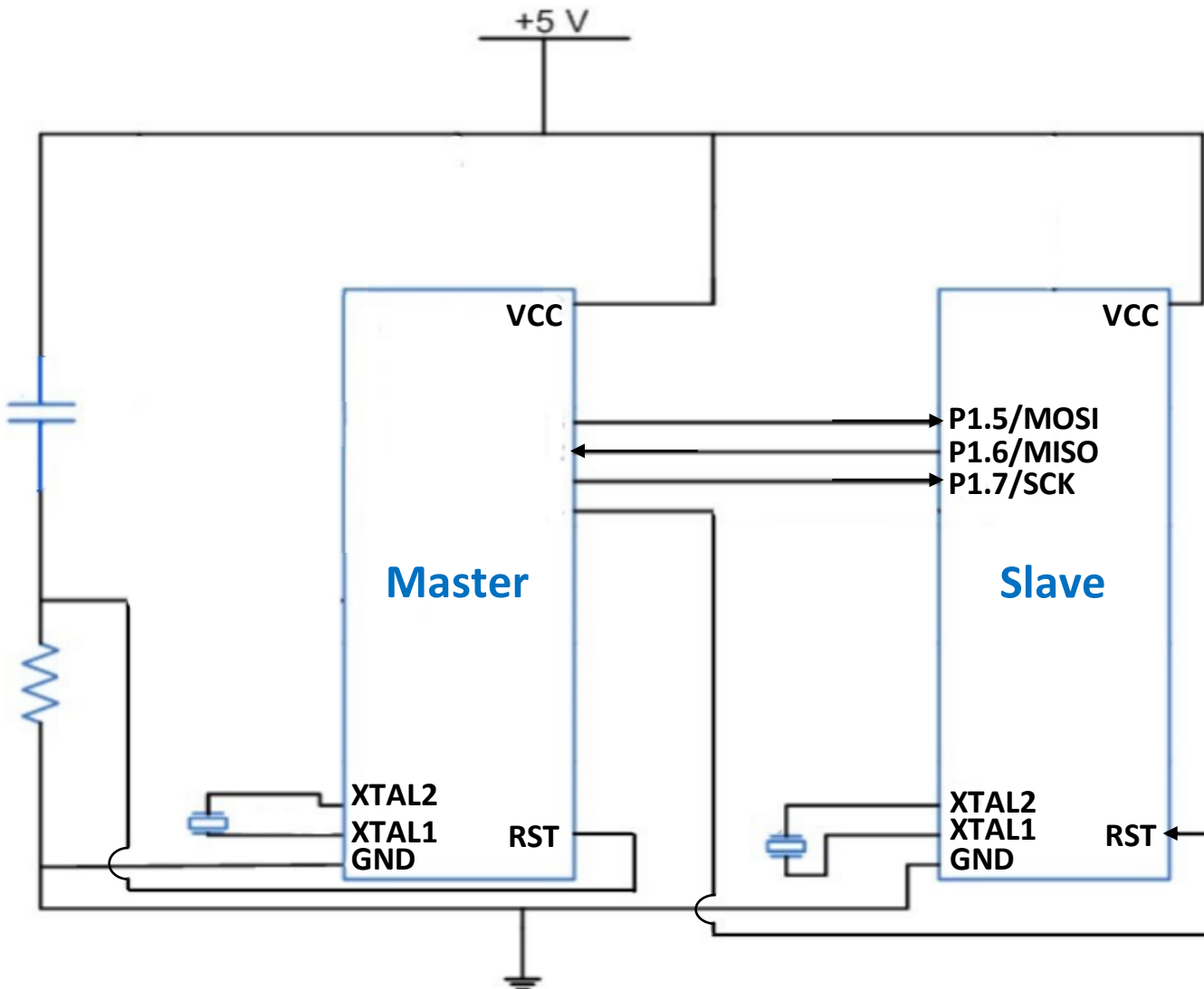


8051 Memory I/O

- MOSI: Master Out Slave In
 - This is the input pin of the target microcontroller.
 - Every signal including data, address and control are received on this pin of the target microcontroller.
- MISO: Master In Slave Out
 - This is the output pin of the target microcontroller which sends the acknowledgement signals to the master controller.



8051 Memory I/O



P1.0	1	40	VCC
P1.1	2	39	P0.0 (AD0)
P1.2	3	38	P0.1 (AD1)
P1.3	4	37	P0.2 (AD2)
P1.4	5	36	P0.3 (AD3)
(MOSI) P1.5	6	35	P0.4 (AD4)
(MISO) P1.6	7	34	P0.5 (AD5)
(SCK) P1.7	8	33	P0.6 (AD6)
RST	9	32	P0.7 (AD7)
(RXD) P3.0	10	31	$\overline{EA}/VPP$
(TXD) P3.1	11	30	ALE/ $\overline{PROG}$
($\overline{INT0}$) P3.2	12	29	$\overline{PSEN}$
($\overline{INT1}$) P3.3	13	28	P2.7 (A15)
(T0) P3.4	14	27	P2.6 (A14)
(T1) P3.5	15	26	P2.5 (A13)
($\overline{WR}$) P3.6	16	25	P2.4 (A12)
($\overline{RD}$) P3.7	17	24	P2.3 (A11)
XTAL2	18	23	P2.2 (A10)
XTAL1	19	22	P2.1 (A9)
GND	20	21	P2.0 (A8)

AT89S51 I/O

Serial Programming Algorithm

1. Power-up sequence:

- Apply power between VCC and GND pins.
- Set RST to high.
- If a crystal is not connected across pins XTAL1 and XTAL2, apply a 3 MHz to 33 MHz clock to XTAL1 pin and wait for at least 10 milliseconds.

2. Enable serial programming:

- Send the Programming Enable serial instruction to pin MOSI/P1.5.
- The frequency of the shift clock supplied at pin SCK/P1.7 needs to be less than the CPU clock at XTAL1 divided by 16.

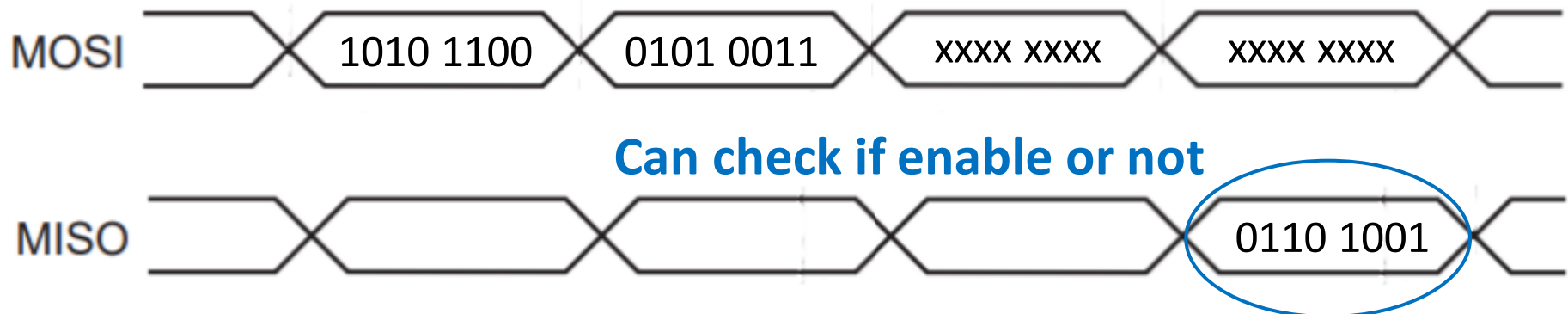
Serial Programming Algorithm

3. The Code array is programmed one byte at a time in either the Byte or Page mode.
 - The write cycle is self-timed and typically takes less than 0.5 ms at 5V.
4. Any memory location can be verified by using the Read instruction that returns the content at the selected address at serial output MISO/P1.6.
5. At the end of a programming session, RST can be set low to commence normal device operation.
6. Power-off sequence (if needed):
 - Set XTAL1 to low (if a crystal is not used).
 - Set RST to low.
 - Turn VCC power off.

Programming Enable

Instruction	Byte1	Byte2	Byte3	Byte4
Programming Enable	1010 1100	0101 0011	xxxx xxxx	xxxx xxxx 0110 1001 (Output)

- Enable Serial Programming while RST is high



Programming Enable

```
//=====Program enable=====
```

```
a=0xac; → 1010 1100  
b=0x53; → 0101 0011  
c=0x00; → Don't Care  
d=0x11; → Don't Care  
sendbyte(a);  
sendbyte(b);  
sendbyte(c);  
sendbyte(d);
```

sendbyte()

```
void sendbyte(unsigned char m)
{
    for (bitno=0; bitno<8; bitno++)
    {
        bit1=m/128;
        m=m<<1;
        sendbit();
    }
}
```

1 byte = 8 bits

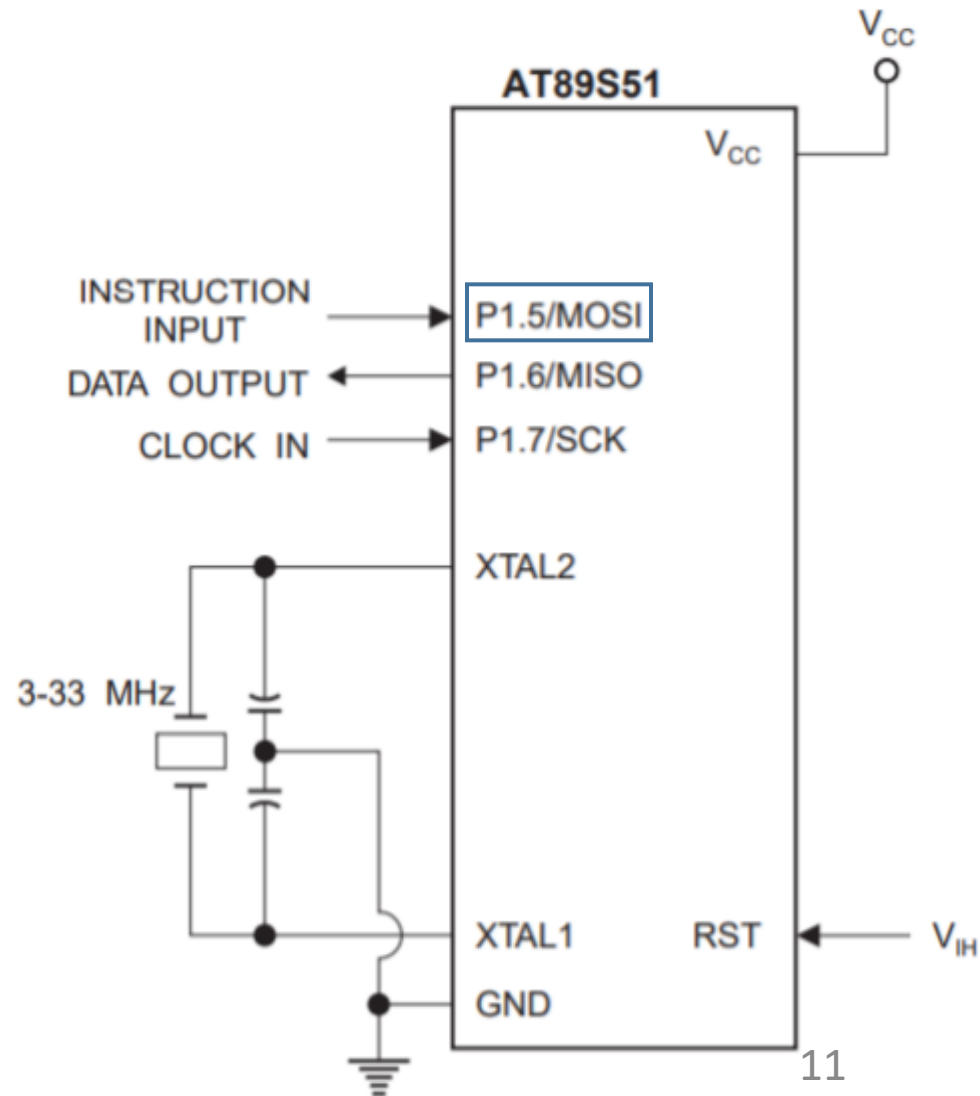
MSB

m<<1

00110011
00110011
00110011
00110011
⋮

sendbit()

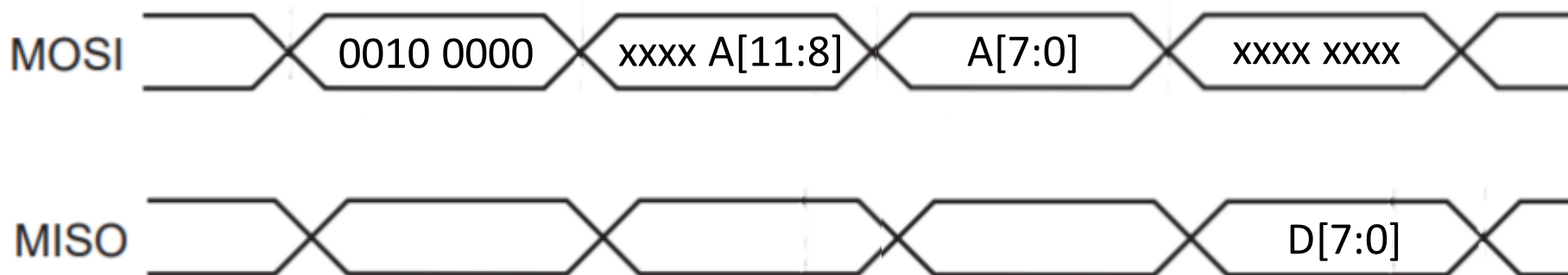
```
void sendbit()  
{  
    mosi=bit1;  
    delay(1);  
    sck=1;  
    delay(1);  
    sck=0;  
}
```



Read Program Memory (Byte)

Instruction	Byte1	Byte2	Byte3	Byte4
Read Memory (Byte Mode)	0010 0000	xxxx A[11:8]	A[7:0]	xxxx xxxx D[7:0] (Output)

- Read data from Program memory in the byte mode
- A[11:0] : 12-bit address , D[7:0] : 8-bit data



Read Program Memory

//=====Read=====

sendbyte (0x20) → 0010 0000

sendbyte (0x00) → xxxx A₁₁A₁₀A₉A₈

sendbyte (0x00) → A₇A₆A₅A₄A₃A₂A₁A₀

sread() ; Read the data at address 0000 0000 0000

sendbyte (0x20) ;

sendbyte (0x00) ;

sendbyte (0x01) ;

sread() ; Read the data at address 0000 0000 0001

sread()

```
void sread()  
{  
    count++;                unsigned char array  
    output[count]=0x00;  
    for (bitno=0;bitno<8;bitno++)  
    {                        1 byte = 8 bits  
        delay(1);  
        sck=1;  
        delay(1);  
        read();  
        sck=0;  
    }  
}
```

read()

```
void read()  
{  
    bit1=miso;  
    output[count]=output[count]<<1;  
    if (bit1==1)  
        output[count]++;  
}
```

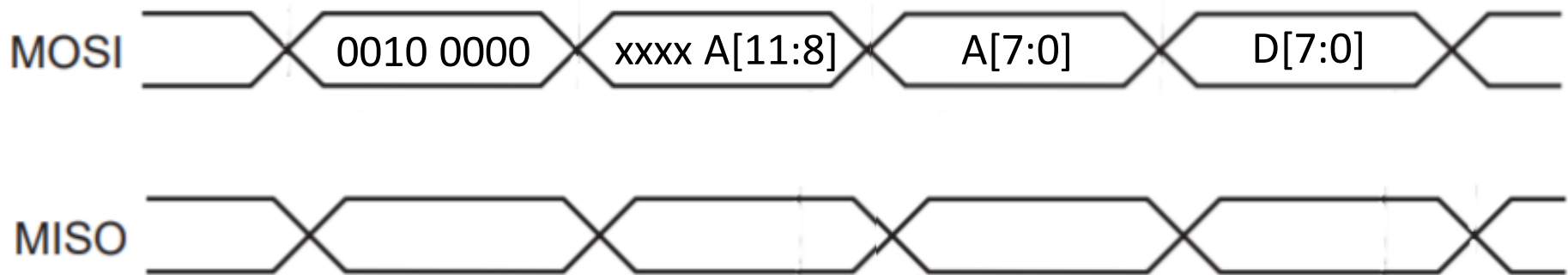
output[count]<<1

00000000
00000000
00100000
00110000
⋮

Write Program Memory (Byte)

Instruction	Byte1	Byte2	Byte3	Byte4
Write Memory (Byte Mode)	0100 0000	xxxx A[11:8]	A[7:0]	D[7:0]

- Write data to Program memory in the byte mode
- A[11:0] : 12-bit address, D[7:0] : 8-bit data



Write Program Memory

//=====write=====

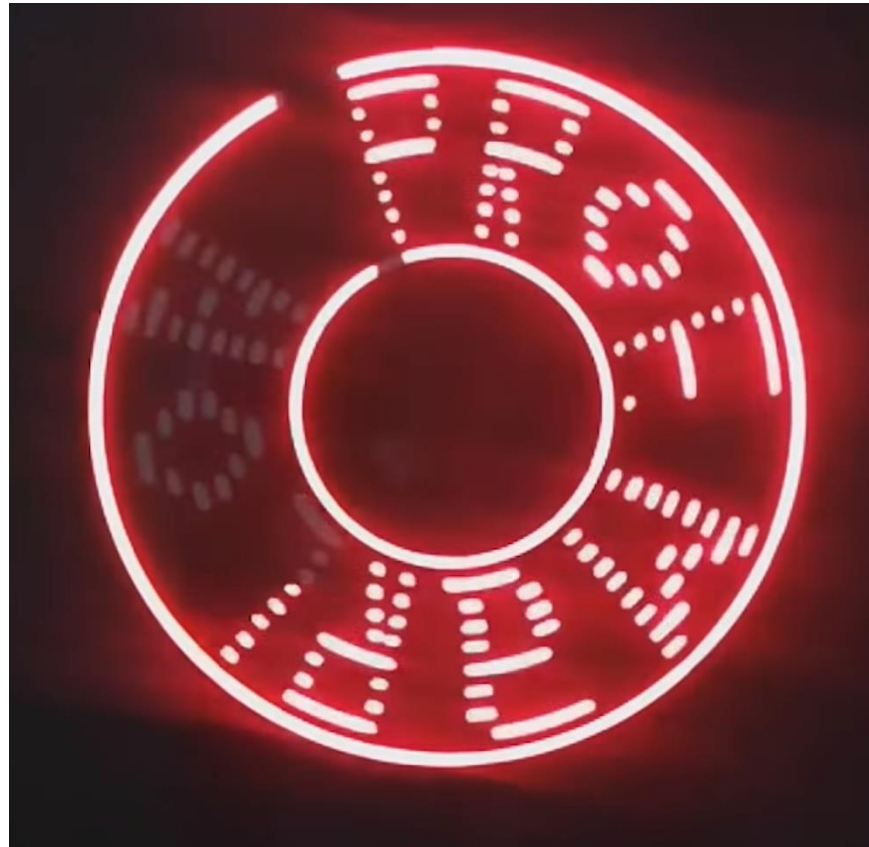
sendbyte (0x40) ; → 0010 0000
sendbyte (0x00) ; → xxxx A₁₁A₁₀A₉A₈
sendbyte (0x00) ; → A₇A₆A₅A₄A₃A₂A₁A₀
sendbyte (0xEE) ;

Write data 1110 1110 at address 0000 0000 0000

sendbyte (0x40) ;
sendbyte (0x00) ;
sendbyte (0x01) ;
sendbyte (0x12) ;

Write data 0001 0010 at address 0000 0000 0001

Persistence of Vision Clock (POV clock)

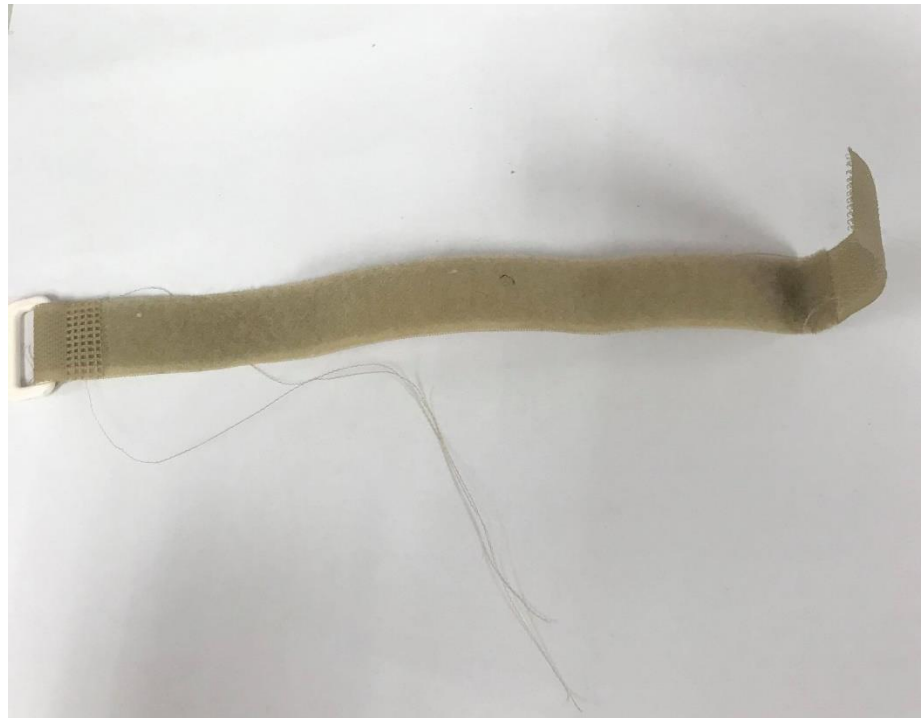


Required Components

Plastic Bottle



Velcro Strap



Required Components

AC/DC Switching Adaptor



POV Module prepared by TA

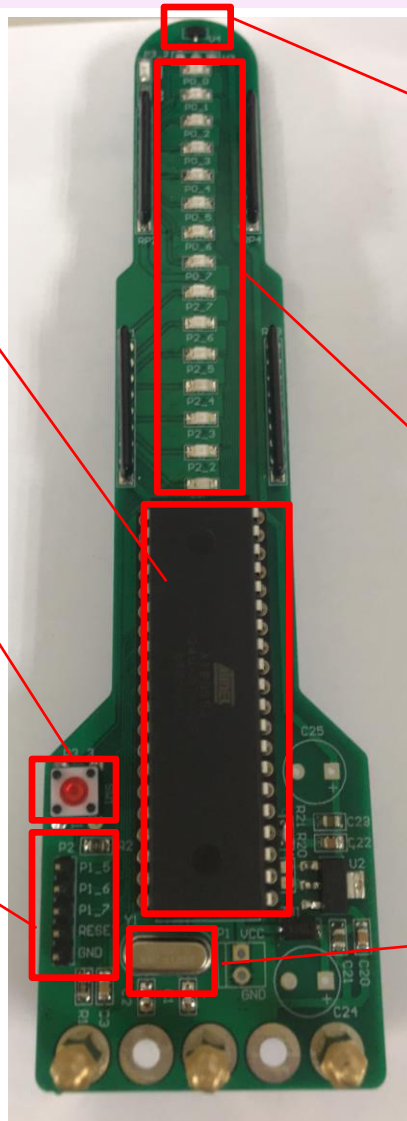


POV Clock Module

- Microprocessor AT89S52

- P3_3 Button for user use

- 5 pins for programming
- MOSI (P1_5)
- MISO (P1_6)
- SCK (P1_7)
- RST
- GND

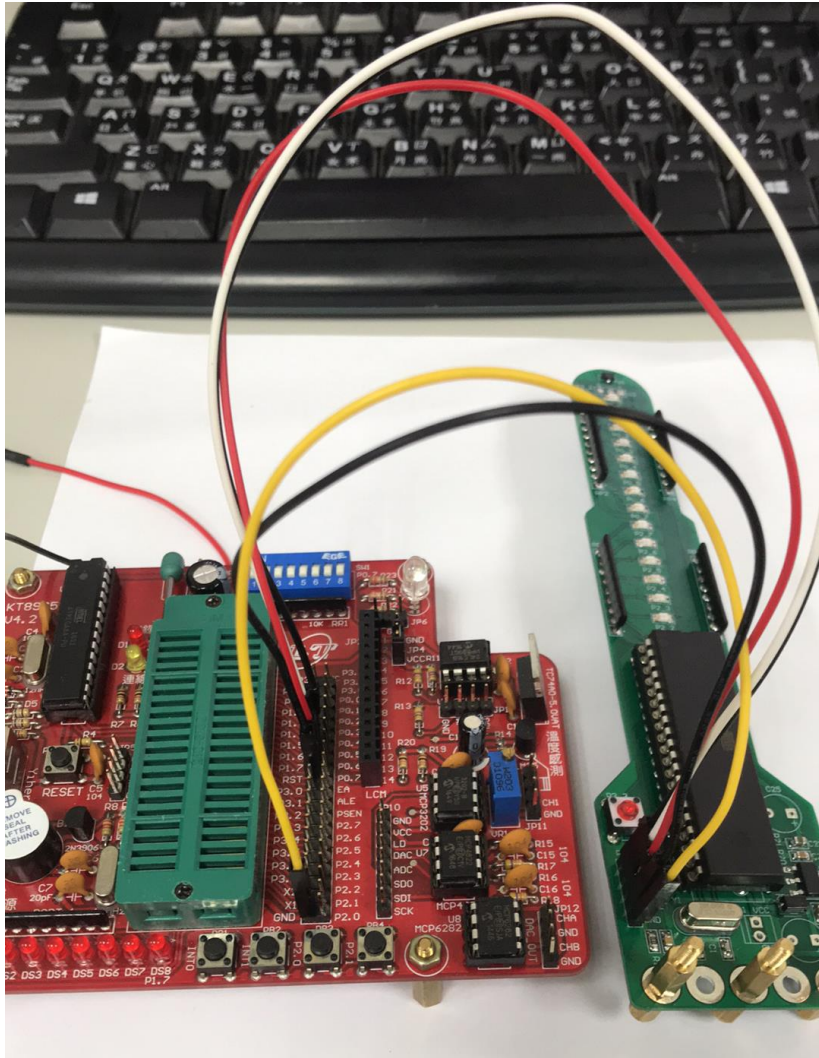


- Hall sensor
- P3_2

- 15 LED on both sides
- P0_0 ~ P0_7
- P1_0 ~ P1_7
- P2_0 ~ P2_7
- P3_0 ~ P3_1
- P3_4 ~ P3_7

- 32Mhz Crystal Clock

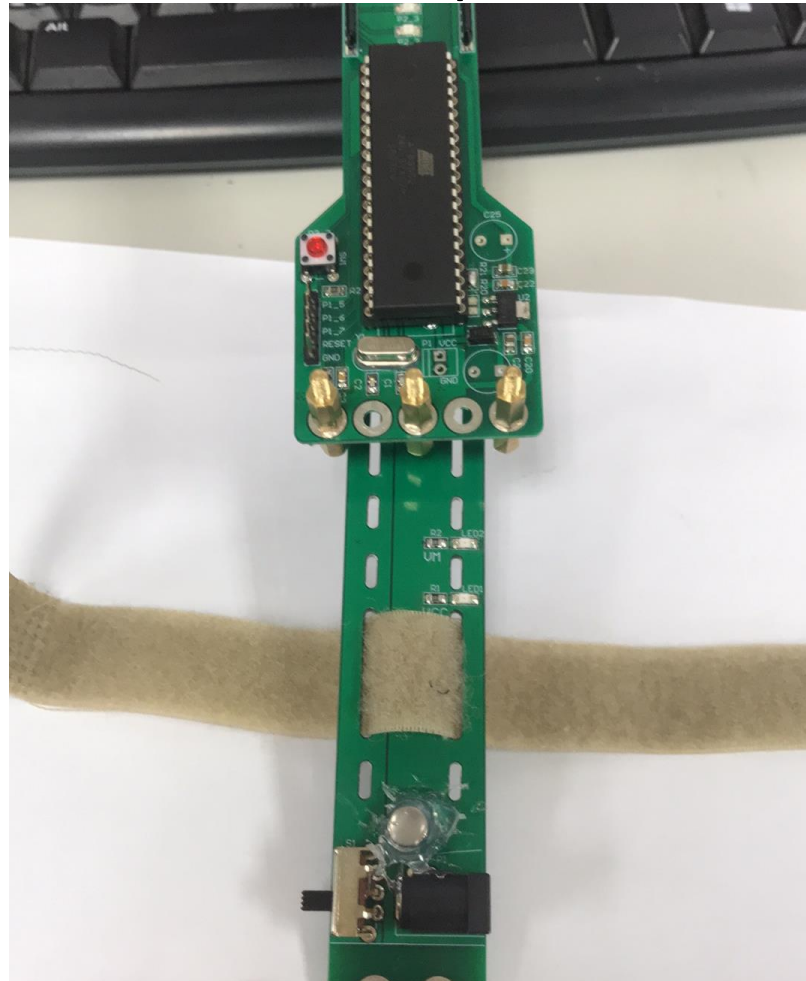
Programming code in POV module



- Remove 8051 on the board first
- Connect 5 pins to programming board
- MOSI (P1_5)
- MISO (P1_6)
- SCK (P1_7)
- RST
- GND
- Using 89S51燒錄軟體 to program code (same as before)

Assemble the Components

- Step1: Fasten velcro strap on the POV module



Assemble the Components

- Step2 : Fasten the POV module on plastic bottle



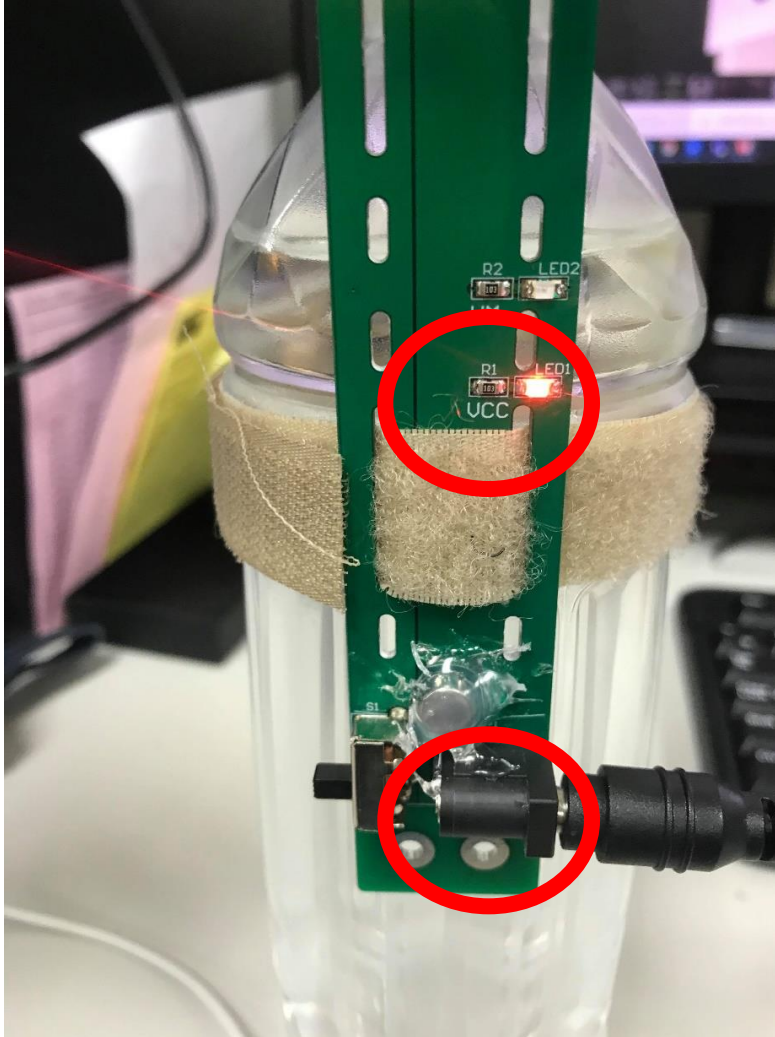
Remember to fasten tightly for safety!

Assemble the Components

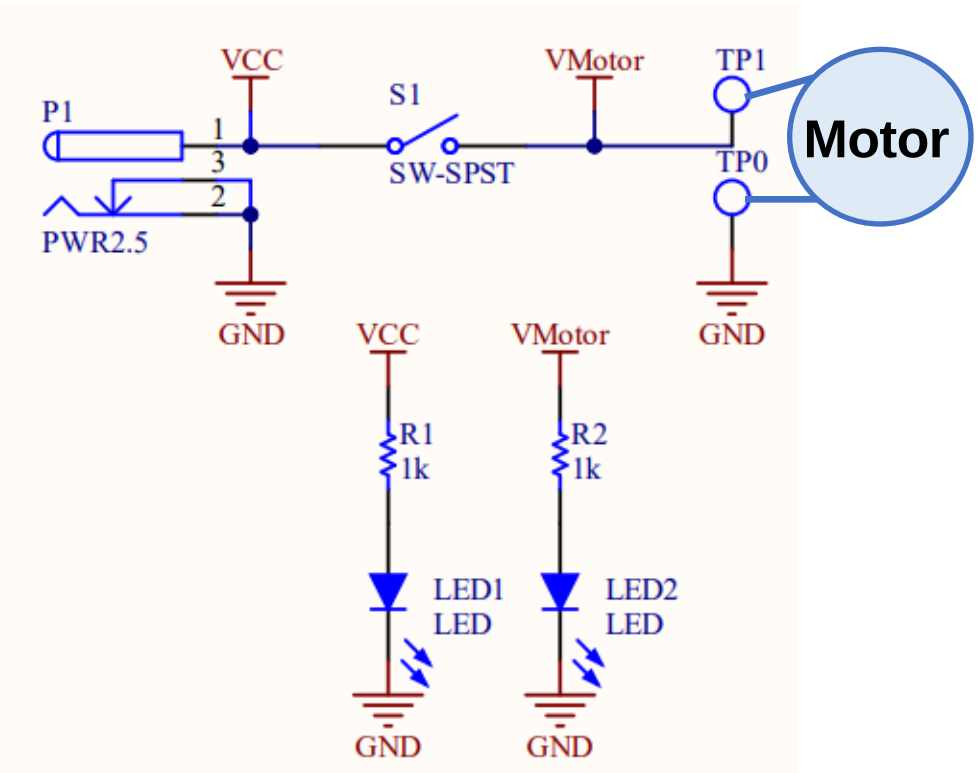
- Step3: Connect power supply and POV module through switching adaptor



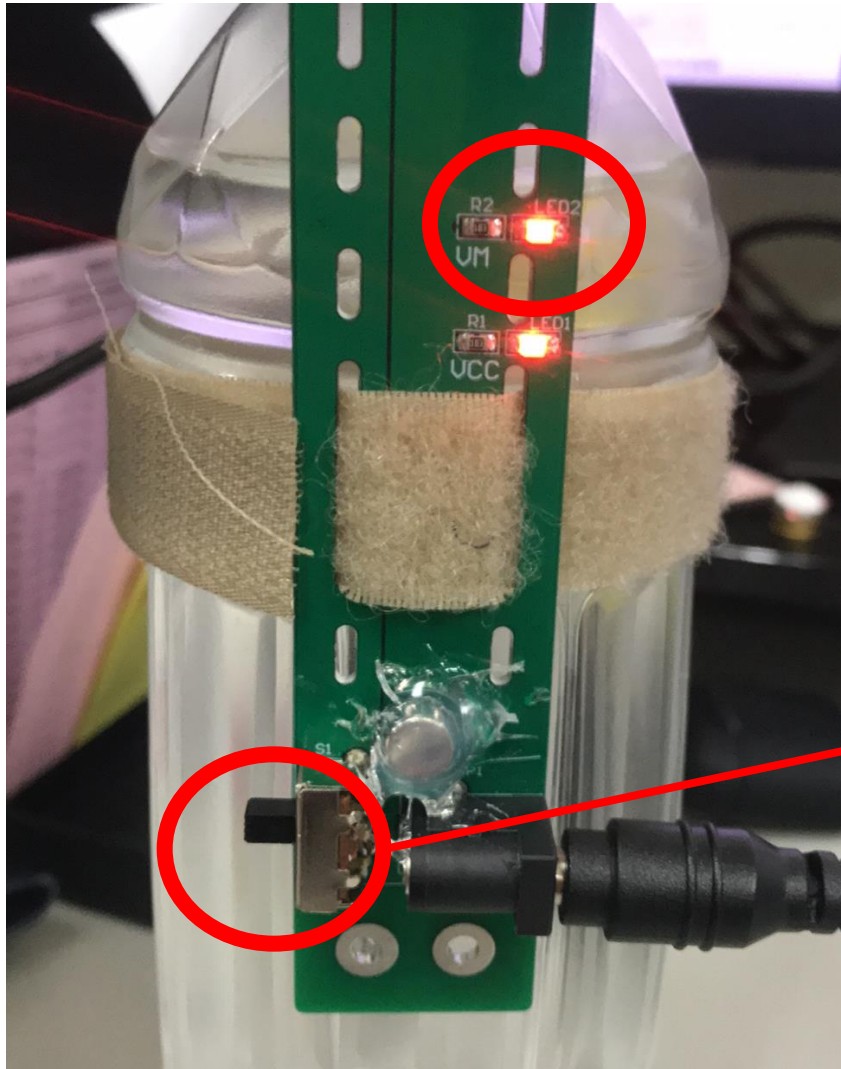
Alert LED



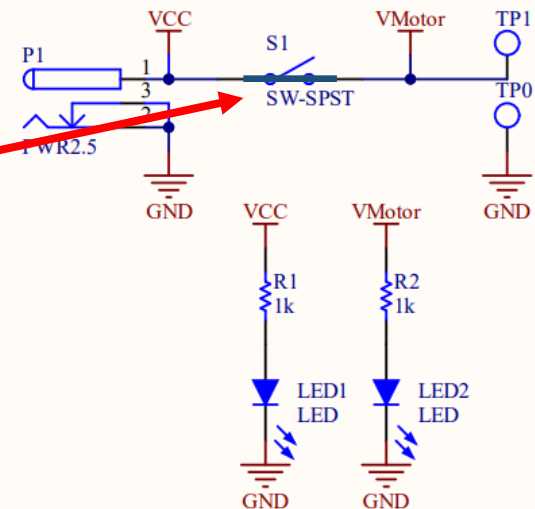
- LED1 will light up if the module is powered up



Alert LED

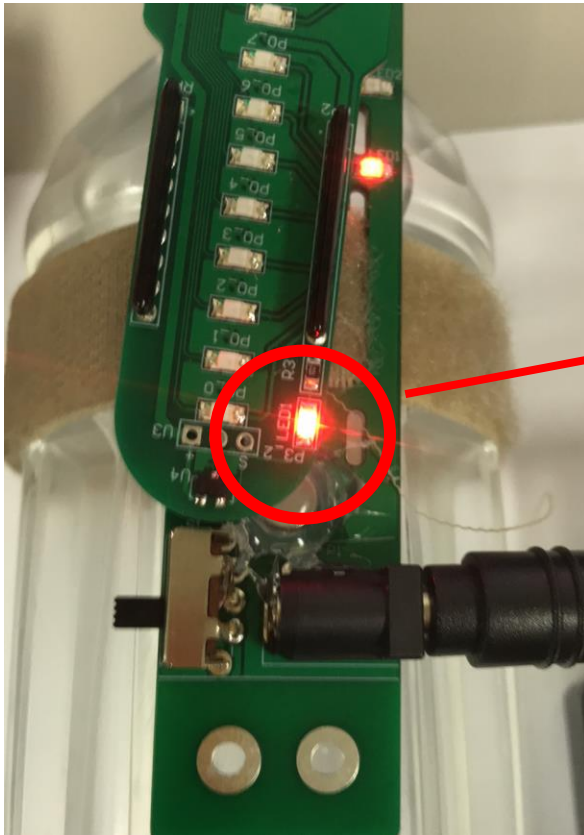


- LED2 will light up if the motor switch is turned on
- **Be careful of your hands!** The motor is turning now.



POV Clock

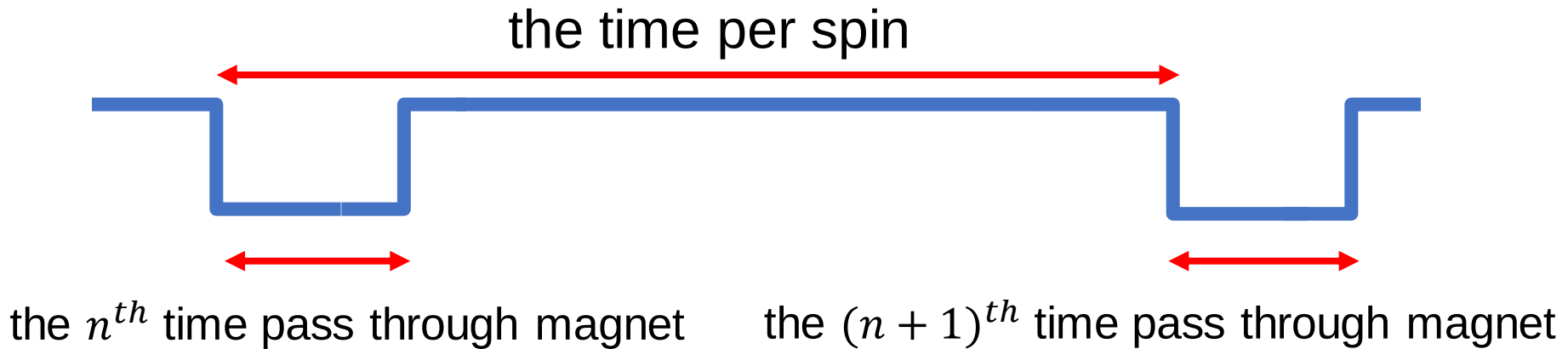
- Using hall sensor to record the time per spin



LED1 will light up whenever the hall sensor pass through the magnet.

POV Clock

- P3_2



- Using timer to record time between 2 passes.
- Determine the position where you want the led light up by calculating the timing.

Reference

- https://www.keil.com/dd/docs/datashts/atmel/at89s51_ds.pdf
- https://www.keil.com/dd/docs/datashts/atmel/at89s52_ds.pdf
- <https://www.engineersgarage.com/tutorials/89s51-52-isp-programmer-basics/>